CSCI435

Project Report

Spring’25

TEAM MEMBERS

Name	Student ID
Taha Hussain (Leader)	7714075
Guzail Rafi	7926972
Sharen Thomas	7700787
Muhammad Abdul Mohsin	8703163
Kavya Vinod	7008995
Vyshnav Vinod	8448462

SUBJECT COORDINATORS

Name	Role
Dr. George Tsaramirsis	Professor
Ms. Asma Damankesh	Tutor

TABLE OF CONTENTS

INTRODUCTION **5**
IMPLEMENTATION **6**
1. Dataset Creation 6
2. Time-of-Day Classification 6
❖ Input and Output 7
❖ Testing & Iteration 9
3. Landmark Detection and Recognition 9
3.1 Classifier and Template Loading 9
3.2 Image Input 9
3.3 Object Detection 10
3.4 Custom Bounding Logic 10
3.5 Failure Handling 11
3.6 Display and Scaling 11
4. Face Detection 11
4.1 Input 11
4.2 Classifier Setup 12
4.3 Face and Feature Detection 12
4.4 Mood Estimation 12
4.5 Annotation and Display 12
4.6 Failure Handling 13
5. Interface for Building and Face Detection 13
5.1 Dual Detection Modes 13
5.2 Image Navigation & Annotation 13
5.3 Quality Metrics Analysis 14
5.4 Gallery & Detection Output 14
5.5 Additional Tools 14
❖ Why HAAR Cascade 14
6. Similarity Retrieval 15
7. Image Quality Assessment 18
8. Annotation Capability 19
9. Integration 20
CHALLENGES FACED **21**
REFERENCES **23**
CONCLUSION **23**

WORK DISTRIBUTION.....25
Code File.....26

INTRODUCTION

Dubai is recognized for its distinctive skyline, impressive city architecture, and vibrant communal spaces, offering an ideal environment for exploring real-world uses of computer vision. This initiative introduces a 'Visual Landmark and Scene Analysis Application' that utilizes OpenCV to examine and understand images captured within the city. The application is designed to detect and categorize different visual elements, such as architectural landmarks, individuals, animals, and everyday objects, improving the understanding of city environments through automated interpretation.

The model focuses on 6 main functions: Dataset creation, landmark and face detection, time-of-day classification, image quality assessment, similarity retrieval and manual image annotation and newly added feature, which is object and mood detection function. These functionalities enable users to gain meaningful insights from visual data by examining individual images or videos. Object and facial recognition help to identify the key elements within an image, while classification methods determine whether the image was captured during daylight or nighttime. Moreover, features such as similarity-retrieval and manual annotations provide enhanced interaction and analysis.

All functionalities are developed using OpenCV, and with the techniques explored during lab and tutorial sessions are incorporated throughout the application, supported by extra methods where needed. This report document highlights the implementation of each function , offering brief technical descriptions, followed by an overview of the challenges encountered during the development process, and concluding with an evaluation of the outcomes and future improvement enhancements. Through the integration of multiple image analysis functions, this application showcases the overall capabilities of computer vision.

IMPLEMENTATION

1. Dataset Creation

To support the development of the 'Visual landmark and Scene Analysis Application', a tailored image dataset was assembled through extensive manual effort. More than 130+ images were gathered from various online sources, they were selected carefully, cropped, and then modified to ensure precise detection by the system. Every image was sorted into a primary dataset folder which contains subfolders which are grouped such as Burj Khalifa, Burj Al Arab, Palm Jumeirah, Random buildings, People, Detection People and Persons Pictures. This organized dataset was essential for developing and testing all 7 main functions of the application, including time-of-day classification, landmark detection and recognition, face detection, similarity retrieval, image quality assessment, annotation capability and even object and mood detection based on facial expressions of the people. There was also a live session that detects people and the objects.

The application provides an image uploading option, which is to upload a folder of images. When an entire folder is uploaded, each image is shown one by one about a one-second interval, simulating a frame-by-frame analysis experience. Importantly, there is no requirement to manually load datasets in advance, as the images are directly uploaded and processed through the user interface. This adaptable and user-friendly mechanism enables real-time testing and demonstration of all app functions, including landmark and facial detection and mood analysis.

2. Time-of-Day Classification

This component of the program is designed to analyze images and determine whether they were taken during the day or at night.
The classification process utilizes OpenCV, beginning with a

conversion of the input image to grayscale. Then, the key decision-making step involves examining the distribution of light and dark pixels.

If an image has higher overall brightness and a significant concentration of light pixels, it is classified as "Day." In contrast, images with lower brightness levels and a dominance of dark pixels are categorized as "Night." This approach was selected after thorough testing and demonstrated the highest accuracy in meeting the project's classification criteria.

Although other methods were taken into consideration such as deep learning modules and histogram equalizer, they were not used as they were demanding too much computing for a simple classification task. This led to the conclusion of using grayscale pixel intensity analysis as it's efficient and effective.

Additionally, grayscale analysis is a very common way used for classification where colors are not the primary factor. It makes it easy to compute while retaining luminance, which is a key factor for detecting different light conditions between day and night (Gonzalez and Woods, 2018).

❖ Input and Output

Input

The module offers a method for supplying images:

- **Directory Input**: Users can specify a folder containing multiple images, and the system will process and classify all compatible files in that folder.

Output

Each image is assigned one of the following labels after processing:

- **Day**: Indicates the image was likely taken during daylight.

- **Night**: Indicates the image was likely taken at night.
- **Error**: Assigned when the image cannot be processed (e.g., due to file corruption or unsupported format).

When a directory is used as input, the system automatically organizes the images into “Day,” “Night,” and “Error” subfolders inside a newly generated sorted_images directory located next to the original input folder.

❖ Code

It's evident from the script that a modular approach was used to program this application. The fundamental functions are broken down into classes that utilize Tkinter for the graphical user interface, with the core logic leveraging OpenCV for image processing.

The main classification logic is contained within the ‘classify_image’ function. As stated in this document, this function is responsible for both input and output: It takes file paths as input and outputs the determined classification (Day, Night, or Error).

The process begins by loading the images and converting them to grayscale, followed by brightness analysis achieved by removing color information. After this, the function computes several critical metrics: the average pixel intensity across the image, the proportion of very dark pixels (those below a defined low-intensity threshold), and the proportion of very bright pixels (those exceeding a higher intensity threshold).

Subsequently, the function applies a set of conditions that characterize the image, classifying it as either Day or Night based on the presence of bright or dark pixels.

- **Testing & Iteration**

  Testing involved processing various batches of the dataset in iterations and continuously evaluating the classification accuracy. Initially, the classification logic was based on HSV color space, and then it was evolved to grayscaling. With our testing, grayscale analysis had a more efficient and effective logic in classification for distinguishing between day and night images. It tested highest in accuracy and consistently met project requirements.

3. Landmark Detection and Recognition

Positive samples of three landmark buildings—Burj Al Arab, Jumeirah Beach Hotel, and Burj Khalifa—were gathered alongside a diverse set of negative samples that did not contain any of these structures. Initial manual annotation attempts using traditional tools led to bounding box inconsistencies. To address this, a GUI-based HAAR Cascade Training Tool was employed. This interface allowed us to annotate, train, and preview detection results interactively, significantly improving the accuracy and robustness of the final XML classifiers.

3.1 Classifier and Template Loading

- HAAR Classifiers were loaded using 'cv2.CascadeClassifier()'. Validation checks ensured each XML file was successfully initialized.

- ORB Templates were processed using a custom ORB detector class that extracted keypoints and descriptors from reference images for each landmark.

3.2 Image Input

All images in the target dataset were loaded from the given folder and converted to grayscale. HAAR detection relies on intensity gradients, while

ORB operates on grayscale features, making this step essential for both techniques.

3.3 Object Detection

- For HAAR detection, detectMultiScale() was used with fine-tuned parameters:
    - scaleFactor = 1.03 to control the pyramid scaling,
    - minNeighbors = 2 for minimal neighborhood validation,
    - minSize = (20, 20) to ignore small noise.
- For ORB detection, the query image's keypoints were matched against template descriptors using KNN matching and Lowe's ratio test, followed by RANSAC-based homography estimation to determine if a landmark was present.

3.4 Custom Bounding Logic

- HAAR Detection: If multiple regions were detected for a single structure, the topmost and bottommost (or leftmost/rightmost for JBH) bounding boxes were merged to form a single large rectangle to properly frame the building.
- ORB Detection: A bounding rectangle was estimated from the homography-transformed template shape and returned if sufficient inliers were found.

All detected landmarks were labeled using cv2.putText() with a black shadowed white font to ensure clear visibility.

3.5 Failure Handling

- If no detection was made by either HAAR or ORB, a red "No Detection" label was overlaid.

- This dual-strategy approach (HAAR + ORB) helped improve robustness, especially in cases where HAAR missed detections due to lighting or scale variation.

3.6 Display and Scaling

- Output images were resized proportionally to fit the viewing window.
- Images were displayed using cv2.imshow() in a resizable window created with cv2.WINDOW_NORMAL.

4. Face Detection

In our implementation, face detection is not just about drawing rectangles around faces, it goes a step further by analyzing facial features and expressions. Once a face is detected within the scene, the system attempts to locate eyes and smiles within that face region. This multi-level detection provides a deeper understanding of the subject's mood and expression.

4.1 Input

The face detection system supports multiple input types, offering flexibility for various use cases. Users can supply:

- A **video file** for continuous face tracking across frames, or

- A **folder containing multiple images** to perform batch processing on a collection of target faces.

4.2 Classifier Setup

Three separate Haar cascades were used for different facial features:

- ‘haarcascade_frontalface_alt.xml’ for face detection.
- ‘haarcascade_eye_tree_eyeglasses.xml’ for detecting eyes.
- ‘haarcascade_smile.xml’ for identifying smiles.

These were loaded using ‘cv2.CascadeClassifier()’, with error handling in place to validate successful loading.

4.3 Face and Feature Detection

- The face was first detected using ‘detectMultiScale()’ with optimized parameters.
- For each detected face, eye and smile detection were applied inside the face detection region to identify key expressions.

4.4 Mood Estimation

If a smile was successfully detected in the lower half of the face, the person’s mood was labeled as "Happy". Otherwise, the mood defaulted to "Neutral".

4.5 Annotation and Display

The GUI will prompt the user to select either a single photo/video or a folder containing couple of photos

After selection:

- Detected faces were highlighted in blue
- Eyes were marked in green
- Smiles were shown in red

- The mood label was displayed above the face using 'cv2.putText()' with both black and white outlines for enhanced visibility

4.6 Failure Handling

In cases where no face was detected within a person's bounding box, no facial features were annotated, and mood inference was skipped.

5. Interface for Building and Face Detection

5.1 Dual Detection Modes

- Supports detection of **faces, eyes, and smiles** using Haar cascades.
- Detects **buildings or people** using custom Haar classifiers.
- Users can independently enable/disable each detection type.

5.2 Image Navigation & Annotation

- Allows folder-based image browsing with Previous/Next buttons.
- Annotation tools include **rectangle, line, circle**, and **freehand** with color selection.
- Options to **save** or **clear** annotations are available.

5.3 Quality Metrics Analysis

- Displays **brightness, contrast, sharpness**, and **day/night** classification.
- Provides enhancement suggestions if quality is poor.
- Includes an RGB histogram for pixel intensity distribution.

5.4 Gallery & Detection Output

- Scrollable gallery for image previews.
- Detections are shown with labeled bounding boxes and stats (e.g., smile count, processing time).

5.5 Additional Tools

- **Live video mode** for real-time detection.
- **Full-screen mode** for better visualization.
- **Find Similar Images** feature based on HSV histogram comparison.

❖ Why HAAR Cascade

Haar Cascade classifiers offer several advantages that make them particularly suitable for real-time and resource-constrained applications:

1. **High Processing Speed**
   Haar cascades are highly efficient in terms of computation. Their use of integral images and a cascaded decision-making structure allows

for rapid object detection, enabling real-time performance even on low-power hardware such as embedded systems or standard laptops.

2. **Effective for Detecting Simple, Structured Objects**
   These classifiers are well-suited for detecting objects with clearly defined and consistent features, such as human faces, eyes, and frontal views of vehicles. Their performance is especially reliable in scenarios involving structured and predictable patterns.

3. **Ease of Implementation**
   Haar-based detectors are straightforward to use, especially within frameworks like OpenCV, where pre-trained models are readily available. Additionally, they can be trained with relatively small datasets, making them accessible for educational, research, and prototype-level applications.

4. **Low Resource and Memory Footprint**
   Haar cascades require minimal memory and computational resources, making them an excellent choice for deployment on devices with limited storage or processing capabilities.

While Haar Cascades may not match the flexibility or accuracy of modern deep learning models in complex or variable environments, they remain a practical and effective solution for lightweight, real-time detection of simple objects. (Gino, 2023)

6. Similarity Retrieval

We built a system that compares one image with many others and finds the **top 3 most similar images** based on either:

- **Color patterns** (using HSV color histograms), or

- **Edges and outlines** (using edge detection).

This system helps in finding images that look alike or share common features.

Comparing images automatically is useful for:

- Detecting duplicates or near-duplicates in large image collections.
- Grouping similar images.

Here's how the system works step by step:

7.1 Feature Extraction

We use two ways to understand what's in an image:

- **HSV Histogram (Color-based)**:
    - Converts the image to HSV (Hue, Saturation, Value) format.
    - Measures how often certain color combinations appear.
    - Great for comparing overall color patterns in images.
- **Edge Histogram (Structure-based)**:
    - Detects edges (like outlines of shapes or objects) in the image.
    - Looks at how frequently different edge intensities appear.
    - Useful for comparing image structure, especially when colors vary.

7.2 Measuring Similarity

We compare the features (color or edge data) of two images using different methods:

- **Correlation** – checks how similar the patterns are.
- **Chi-Square** – checks how different they are (used with a negative sign so higher means more similar).
- **Intersection** – checks how much overlap exists between two histograms.
- **Bhattacharyya** – measures distance between histograms (also used negatively).

The result is a similarity **score**: higher means more similar.

7.3 Matching and Ranking

The system:

- Reads the input image.
- Reads each candidate image one by one.
- Extracts features from each image.
- Compares them to the input image.
- Stores the image paths with their similarity scores.

Finally, it sorts all the candidate images by score and returns the **top 3 most similar ones**.

The system handles errors like missing files or unreadable images.

It lets you choose whether to compare images using color (`hsv`) or edges (`edges`).

It also lets you select which similarity method to use.

This kind of system is helpful in many real-world applications like:

- Finding similar-looking photos.
- Image search engines.
- Detecting reused content.
- Sorting images in a smart way.

7. Image Quality Assessment

In this part of the system, we check how good or bad an image is by looking at three main things:

1. **Brightness:** How light or dark the image is.
2. **Contrast:** The difference between light and dark areas
3. **Sharpness:** How clear or burry the image looks

The purpose of this system is to

1. Automatically evaluate a large collection of images.
2. Spot poor-quality images that might need editing or replacement.
3. Provide clear and measurable quality scores instead of replying only on visual judgement.
4. Ensure consistent image quality across multiple images when processing in bulk.

We used Python, along with OpenCV and NumPy libraries, to perform the analysis.
The following happened:

- The system read the images and converts it into greyscale to simplify analysis
- Then it calculates:
    - Brightness: by finding the average of all pixel values.
    - Contrast: by checking how spread out the pixel values are.
    - Sharpness: by using a technique that detects how much detail the image contains.

7.1 Quality Standards Used

To check if the image is good quality or not we have set some rules

Metric	What it Shows	Threshold
Brightness	Lightness of the image	< 60 = Too dark > 190 = Too bright
Contrast	Light vs Dark difference	< 30 = Low contrast (flat image)
Sharpness	Image clarity	< 100 = Blurry or out of focus

This system helps ensure all images are checked in the same way and only good-quality images are used or saved.

8. Annotation Capability

We built a flexible system that lets users **draw on images**. With this system, users can:

- Draw shapes like **rectangles, circles, lines**, or even **freehand**.
- **Choose colors** and **adjust line thickness** to their liking.
- **Save** the annotated images while **keeping the original image unchanged**.
- **Clear** all drawings and start over whenever needed.

Adding annotation features is important because:

- It helps users **highlight specific parts** of an image.
- It's useful for **pointing out quality issues** in a visual way.
- It supports **machine learning**, where marked images are often needed to train models.
- It allows users to **visually explain their analysis** or findings.

The system keeps track of what the user draws in two ways:

1. **On the canvas** – So users can instantly see what they're drawing.

2. **On the actual image** – By saving the shape's location using exact image coordinates.

9. Integration

We designed a complete graphical user interface (GUI) for the application that includes:

- A **main window** that acts as the central hub for the app.
- Separate windows for **image analysis** and **image annotation**.
- A **batch results viewer** to show the analysis results of multiple images at once.
- A **consistent look and feel** across all parts of the application.

We created this integrated GUI to:

- Make the app easy to use, even for tasks that are technically complex.
- Allow users to access different features (analysis, annotation, etc.) from one place.
- Keep each feature in its own section so the app feels organized and modular.
- Provide a clean, professional appearance with matching styles and behaviors.

In code the integration works this way:

7.1 Window Management

When a user clicks to open a tool like the image analyzer:

- The system **checks if the window is already open**. If not, it creates a new one.
- If the window is already open but hidden, it **brings it to the front** so the user can see it.
- This ensures that **only one version of each tool is open** at a time, which keeps the interface clean and avoids confusion.

7.2 Data Flow Between Windows

When a user analyzes a batch of images:

- The system **saves the results**.
- Then, it **opens a new window** to show those results.
- This smooth flow of data ensures users always see the latest analysis without needing to manually transfer anything.

7.2 Consistent Styling

To make the app look polished and professional:

- We used a predefined theme that provides a clean base look.
- We applied **custom styles** to elements like frames and buttons—for example, using a green color for action buttons with white text.
- This makes the app more visually appealing and easier to navigate.

CHALLENGES FACED

We, as a team faced quite a lot of challenges, such as the following:

- The manual effort involved collecting a large quantity (130+) of images, which then needed to be resized manually to maintain consistency across the dataset.

- The project required an extensive amount of manual effort, involving 50 to more images per group, which included cropping and then annotating each one was particularly a tedious task when drawing the bounding boxes.
- To obtain reliable outcomes, the model required a balanced set of both positive and negative samples, which increased the workload of data preparation.
- Similar architectural elements such as the curved designs of the Burj Al Arab and Burj Khalifa led to frequent misclassifications and also the model frequently tended to over-detect, like placing several bounding boxes over all buildings in an image of Palm Jumeirah, even though only one was relevant.
- False detections were common, often mistaken Burj Khalifa for Burj Al Arab because of the similar top structures or color schemes.
- In certain cases, such as with Burj Khalifa, the model generated 4 to 5 bounding boxes on the building , which was inaccurate. To address this issue, a manual adjustment was implemented in the code, merging the upper and lower boxes into a single rectangle to ensure proper detection.
- Moreover, the Haar Cascade detection method was outdated, resulting in inaccurate and excessive detections in general.
- The Haar Cascade GUI that was used for training, was very time-consuming, particularly on low-performance (slower) laptops. The duration of training increased sharply with additional steps, the default 20 steps took about 10 minutes, while increasing it to 23 steps took almost an hour, and going beyond 40 steps caused the system to hang or fail to finish. Also, the GUI was resource-heavy, which restricted the use of other applications during training and needing a high-performance laptop to run smoothly.

REFERENCES

Gino, S. (2023). *Custom Object Detection with Haar Cascade (VJ) - Shahar Gino - Medium*. [online] Medium. Available at: https://sgino209.medium.com/custom-object-detection-with-haar-cascade-vj-93c197b59d5b (Accessed 19 June 2025).

Gonzalez, R.C. and Woods, R.E. (2018) *Digital image processing: 4th edition* [PDF]. Available at: https://www.cl72.org/090imagePLib/books/Gonzales,Woods-Digital.Image.Processing.4th.Edition.pdf (Accessed 19 June 2025).

CONCLUSION

The development of the **Visual Landmark and Scene Analysis Application** brought together various image processing techniques to create a powerful and user-friendly computer vision tool. From dataset creation to detection, classification, annotation, and image similarity analysis, the project successfully implemented multiple core functions using OpenCV and Python.

Despite challenges such as heavy manual dataset preparation, annotation efforts, and technical limitations of Haar Cascade classifiers, the team overcame these obstacles through custom logic enhancements, iterative testing, and interface improvements. The integration of features like image quality assessment, facial mood detection, and a batch analysis GUI ensured a cohesive and professional user experience.

Overall, the project demonstrated how classical computer vision techniques can be effectively combined to analyze real-world scenes and objects, offering valuable insights into visual data. It not only met the core objectives but also set a strong foundation for future enhancements, such

as integrating deep learning models or cloud-based deployment. Through teamwork, problem-solving, and innovation, this project highlighted the practical potential of computer vision in smart city applications like those in Dubai.

WORK DISTRIBUTION

Parts	Name	Student ID
Coding Part	**Everyone -** Guzail Rafi Taha Hussain Sharen Thomas Muhammad Abdul Mohsin Kavya Vinod Vyshnav Vinod	 7926972 7714075 7700787 8703163 7008995 8448462
Introduction	Sharen Thomas	7700787
Implementation		
Dataset Creation & Tagging	Sharen Thomas Muhammad Abdul Mohsin	7700787 8703163
Time-of-Day Classification	Sharen Thomas Muhammad Abdul Mohsin	7700787 8703163
Landmark & Face Detection	Kavya Vinod Vyshnav Vinod	7008995 8448462
Similarity Retrieval	Kavya Vinod Vyshnav Vinod	7008995 8448462
Image Quality Assessment	Guzail Rafi Taha Hussain	7926972 7714075
Annotation Capability	Guzail Rafi Taha Hussain	7926972 7714075
Integration	Guzail Rafi Taha Hussain	7926972 7714075
Challenges Faced	Sharen Thomas	7700787
References	Kavya Vinod Sharen Thomas Muhammad Abdul Mohsin	7008995 7700787 8703163

Conclusion	Kavya Vinod	7008995

Code File

https://drive.google.com/file/d/1K69LZso8IDvqF_XzbSx5WdAPPk8j3gUK/view?usp=sharing